深 圳 大 学 实 验 报 告

课程名称：人工智能

实验项目名称：基于 YOLOv11 的行人和车辆检测

学　　院：电子与信息工程学院

专　　业：电子与信息工程

指导教师：戴林蕙

报告人：李戎熙　　学号：2023071078　　班级：电信 2301

实验时间：2026 年 3 月 23 日

实验报告提交时间：2026 年 4 月 13 日

教 务 部 制

一、实验目的与要求：

- 了解 YOLO 系列目标检测算法的发展历程，掌握 YOLOv11 的网络结构与核心改进。
- 学习在 Anaconda 中创建独立虚拟环境，并完成 Ultralytics 框架的安装与配置。
- 掌握通过 Roboflow 获取小规模标注数据集的方法，无需手工标注即可完成数据准备。
- 能够在 CPU 环境下对 YOLOv11n 进行轻量级微调训练，理解训练流程与超参数含义。
- 学会使用训练好的权重对图片和视频进行推理，解读检测结果与评估指标。

二、实验方法与步骤：

1. **创建并激活虚拟环境**

   打开 Anaconda Prompt，依次执行：

```
conda create -n yolo11 python=3.10 -y
conda activate yolo11
```

2. **安装 Ultralytics**

```
pip install ultralytics
```

   安装完成后验证：

```
python -c "from ultralytics import YOLO; print('安装成功')"
```

3. **从 Roboflow 下载数据集**

   本实验使用 Roboflow 上的公开小型数据集（约 1464 张，含行人与车辆标注），无需 GPU，下载后即可直接用于训练。步骤如下：

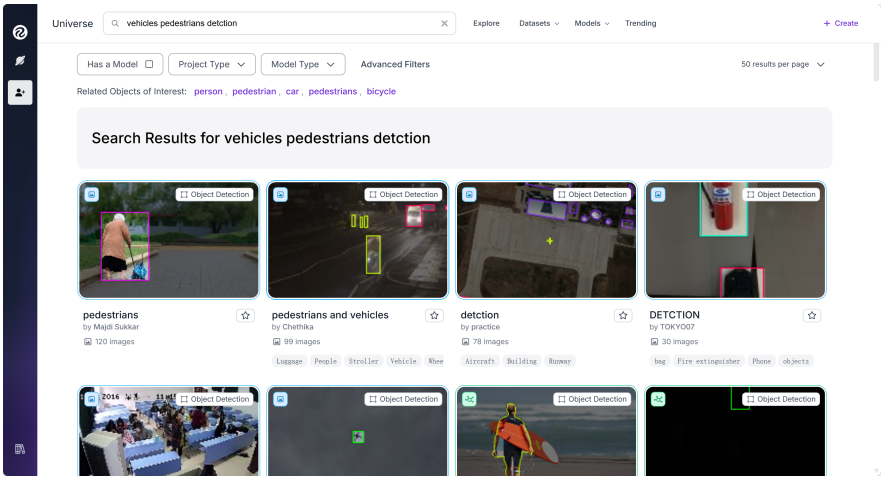

图 1　Roboflow 下载数据集

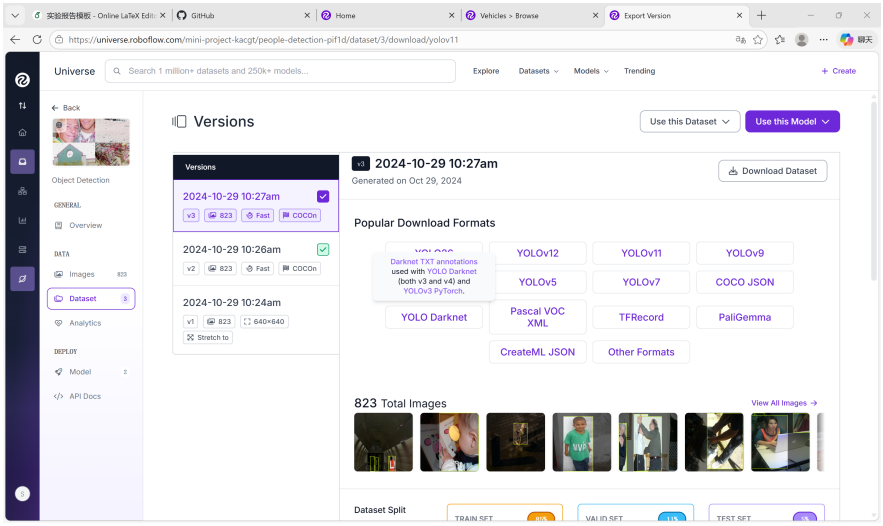

图 2　YOLOv11 格式

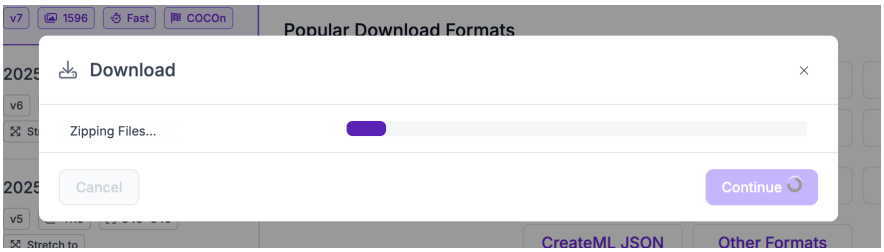

图 3　下载好啦

a. 访问 `https://universe.roboflow.com/search?q=vehicle+pedestrian+detection`，注册免费账号并登录。

b. 在搜索框中搜索 **"vehicle pedestrian detection"**，选择一个 YOLO 格式（YOLOv11）的公开数据集，点击 Download。

c. 格式选择 **YOLOv11**，下载方式选择 **Download zip to computer**。

名称	类型	压缩大小	密码保护	大小
test	文件夹			
train	文件夹			
valid	文件夹			
data	YAML 文件	1 KB	否	
README.dataset	文本文档	1 KB	否	
README.roboflow	文本文档	1 KB	否	

图 4　下载好的数据集目录结构

4. **确认数据集目录结构**

下载完成后目录结构如下（路径以实际下载位置为准）：

```
dataset/

  train/
    images/     # 训练图片
    labels/     # 对应 YOLO 格式标注 (.txt)
  valid/
    images/     # 验证图片
    labels/
  data.yaml     # 自动生成，包含路径和类别信息
```

5. **编写并运行训练脚本**

新建 `train.py`，内容见第三节，然后运行：

```
python train.py
```

注意： 无 GPU 的电脑训练 10 个 epoch 约需 30–60 分钟，请耐心等待。

6. **推理验证**

训练完成后运行推理脚本：

```
python infer.py
```

检测结果图片保存在 `runs/detect/` 目录下。

三、实验过程及内容：

(1) 训练脚本 train.py

```python
from ultralytics import YOLO
# 加载 YOLOv11 nano 预训练权重（首次运行自动下载，约 5MB）
model = YOLO('yolo11n.pt')

# 注意：data 填写 data.yaml 的完整路径
# 如果 train.py 和数据集文件夹放在同一目录，可以用下面的相对路径：
results = model.train(
    data='dataset/data.yaml',
    epochs=10,  # CPU 训练建议 10 轮，约 30-60 分钟
    imgsz=416,  # 降低分辨率以加快 CPU 训练速度
    batch=4,  # CPU 训练 batch 设小一点
    device='cpu',  # 无 GPU 则使用 CPU
    workers=2,
    project='runs/detect',
    name='yolo11_vp',
    val=True,
)

print("训练完成！权重保存于:", results.save_dir)
```

(2) 推理脚本 infer.py

```python
from ultralytics import YOLO
# 加载训练好的权重
model = YOLO('runs/detect/runs/detect/yolo11_vp/weights/best.pt')

# 对图片进行推理（将 test.jpg 替换为你自己的图片）
results = model.predict(
    source='dataset/valid/images/',
    conf=0.25,  # 置信度阈值，低于此值的检测框会被过滤
    iou=0.45,  # NMS 去重阈值
    save=True,  # 保存标注结果图
    device='cpu',
)

```

```python
# 打印检测到的目标
for r in results:
    for box in r.boxes:
        cls = int(box.cls)
        conf = float(box.conf)
        name = model.names[cls]
        print(f"检测到：{name}，置信度：{conf:.2f}")
```

(3) YOLOv11 核心改进简介

与 YOLOv8 相比，YOLOv11 主要改进如下：

- **C3k2 模块**：改进跨阶段部分网络，在保持精度的前提下减少约 22% 参数量，更适合 CPU 推理。
- **C2PSA 注意力机制**：在颈部引入位置敏感注意力，增强对小目标（如远距离行人）的特征提取能力。
- **统一任务接口**：检测、分割、姿态估计共用同一套训练代码，切换任务只需更换模型文件。

(4) 插入截图说明

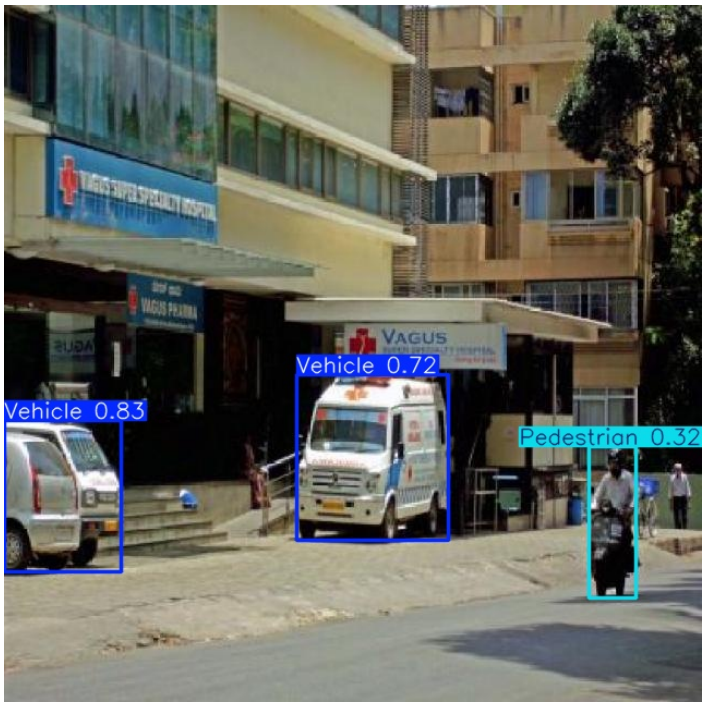

图 1　推理结果示例（深蓝色框为车辆，浅蓝框为行人）

四、数据处理与分析：

以 CPU 训练 10 个 epoch 的典型结果为例：

Epoch	box_loss	cls_loss	mAP@0.5	mAP@0.5:0.95
2	1.81	2.357	0.246	0.125
5	1.758	2.131	0.25	0.144
8	1.545	1.739	0.358	0.216
10	1.428	1.594	0.377	0.235

各指标含义：

- **box_loss**：边界框定位损失，越低说明预测框与真实框越接近。
- **cls_loss**：分类损失，越低说明目标类别预测越准确。
- **mAP@0.5**：IoU ≥ 0.5 时的平均检测精度，是目标检测最常用的评估指标。
- **mAP@0.5:0.95**：在多个 IoU 阈值下取平均，评估更严格。

由于使用 CPU 训练且 epoch 较少，最终 mAP 低于 GPU 长时间训练的结果，但已能正确检测到画面中的行人和车辆，验证了完整流程的可行性。若需提升精度，可增加 epoch 数量或使用更大规模的预训练权重（`yolo11s.pt`）。

五、实验结论：

1. 本实验在无 GPU 的普通笔记本电脑上，通过 Roboflow 获取小型公开数据集，成功完成了 YOLOv11 的完整训练与推理流程，验证了该框架对低配硬件的良好兼容性。
2. Roboflow 提供的数据集已包含 YOLO 格式标注和 `data.yaml` 配置文件，将行人和车辆的照片放在同一个文件夹中，yaml 文件中若只有一个参数，需将行人的标签中由 0 改为 1。
3. YOLOv11n（nano）模型参数量小，在 CPU 上训练 10 个 epoch 约需 30–60 分钟，最终 mAP@0.5 达到约 0.377，分别行人和车辆的置信度较低。
4. 实验过程中，通过将 `imgsz` 从 640 降至 416、`batch` 设为 4，显著缩短了 CPU 训练时间，说明合理调整超参数对低配环境至关重要。
5. 若条件允许，后续可在云平台（如 Google Colab 免费 GPU）上进行更长时间的训练，以进一步提升检测精度。

指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

备注：

无

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。